NLP CS60075 Assignment II

How to read this document. Every project below has a Problem Statement, a list of the three datasets (full technical details for every dataset are in the supporting *Datasets* document), and three colour-coded task boxes: **blue = Task 1 (classical baseline)**, **green = Task 2 (paper reproduction)**, **purple = Task 3 (combine & improve)**. Each task box gives a one-line objective, then numbered step-by-step instructions, then the specific evaluation metrics that task must report.

What to deliver, for every task. For every one of the three tasks in the project you submit: (1) working, runnable code implementing exactly what that task's step-by-step instructions describe; (2) a results table or plot reporting every metric listed under that task's *Evaluation metrics to report*; and (3) a few sentences of discussion connecting those numbers to what the task asked you to investigate. The submitted code must be reproducible and runnable with minimal setup: **either provide a Docker-based environment with a one-command/one-click run, or provide a Google Colab notebook that can be executed end-to-end with minimal manual intervention**. Full code organization, environment, report, and submission-package requirements are given once, in full, in the *Deliverables and Submission Guidelines* section at the end of this document – read it before you start, not after you finish.

Group assignment. Each project lists three datasets. Your group will be assigned exactly *one* of the three datasets for a given project and will have to complete all three tasks (classical baseline, paper reproduction, combine & improve) using only that one dataset – you will not work on the other two.

GPU access. For projects requiring GPU resources, CS students are requested to obtain access to the CSE GPU server, with *one GPU-server account/access request per group*. The completed access-request form ias available at `https://cse.iitkgp.ac.in/archives/Server_Access_Form.pdf` and should be submitted to the Somak sir for approval. Please ensure that the form is complete and accurate before submission.

Project TA. Each group will be assigned one Project TA. The assigned TA's name, email address, and office hours will be provided to the group and should be used for project-related questions, clarification, and support.

Final deliverables. Each group is expected to submit the following:

1. **Code** – complete, working, and reproducible implementation of all three tasks, packaged using either Docker (with a one-command/one-click run) or Google Colab (with an end-to-end runnable notebook).
2. **Report** – a concise report documenting the methodology, experimental setup, results, evaluation metrics, and discussion for all three tasks.
3. **Code walkthrough demo** – a **5–10 minute** walkthrough demonstrating the implementation, explaining the main components of the code, and showing how the experiments can be run.

Final presentation before grading. Before grading, each group must give a **10-minute presentation** of its project to the assigned Project TA. The presentation should briefly cover the problem, approach, key experimental results, comparison across the three tasks, and the main conclusions. The presentation will be evaluated by the Project TA.

Assignment Submission Deadline: November 8, 2026

Project 6: A Text Processing Pipeline for Code-Mixed, Real-World Text

Problem Statement

Code-mixing is the everyday practice of freely mixing two languages within one sentence – e.g. a Hindi-English tweet like "yeh movie bahut acchi thi, must watch!" mixes Romanized Hindi words with English in the same breath. Standard NLP tools assume one language and one script throughout, so they fail immediately here. This project builds a full pipeline to handle exactly this setting, from raw mixed-script text to a sentiment label, on your assigned share of a Hindi-English code-mixed tweet corpus.

Task 1's classical pipeline has three stages: *language identification (LID)* tags each word's language; *normalization* maps the many spelling variants of Romanized Hindi ("acchi"/"achi"/"acha") to one canonical form, using a shared transliteration dictionary (Dakshina) that every group uses identically, regardless of which tweet partition they were assigned; the cleaned text is finally classified with TF-IDF features and a Random Forest. Task 2 reproduces MuRIL, a transformer pretrained specifically on Indian languages and code-mixed text, fine-tuned directly on raw, non-normalized tweets, testing how much of Task 1's normalization work a large multilingual pretrained model absorbs on its own. Task 3 tests whether the two are complementary: feed Task 1's LID predictions into Task 2's model as an extra signal.

Datasets Used *(Full dataset details are in the Datasets document, Section 6.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **SemEval SentiMix – Partition A** — one third of a fixed, seeded 3-way random split of the SemEval-2020 Task 9 Hindi–English tweet corpus. Has its own train/validation/test split, sufficient on its own for all three tasks.
2. **SemEval SentiMix – Partition B** — a second, disjoint third of the same split.
3. **SemEval SentiMix – Partition C** — the remaining third of the same split.

Task 1 — Baseline: Rule-Based LID + Random Forest (30 marks)

Objective. Build a full classical pipeline – language identification, normalization, and sentiment classification – on your assigned partition.

Step-by-step instructions:

1. Language ID: character n-gram ($n = 1$–4) TF-IDF features feeding a logistic regression (L2, $C = 1.0$), 3-way {en, hi, univ}, trained on your assigned partition's train split's gold `lang_tag` labels.
2. Normalization: build a rule-based dictionary for common Romanized-Hindi spelling variants from the shared Dakshina transliteration lexicon (identical for every group, regardless of assigned partition).
3. Downstream sentiment classifier: TF-IDF features (word + character n-grams, $n = 1$–3) over the normalized token sequence, feeding a Random Forest classifier, 300 trees, max depth 20, min samples per leaf 5.
4. Train on your assigned partition's train split; evaluate macro-F1 on its test split.

Evaluation metrics to report:

- Macro-F1 on your assigned partition's test split.

Task 2 — Paper Reproduction: MuRIL Fine-Tuning (30 marks)

Objective. Reproduce a multilingual pretrained model built for Indian-language and code-mixed text, fine-tuned on your assigned partition.

Paper: Khanuja, Bansal, Mehtani, et al., *MuRIL: Multilingual Representations for Indian Languages*, arXiv:2103.10730. **Checkpoint:** `https://huggingface.co/google/muril-base-cased`.

Step-by-step instructions:

1. Fine-tune MuRIL with a 3-way sequence-classification head directly on raw (non-normalized) text from your assigned partition's train split: AdamW, learning rate 2e−5, batch size 16, 4 epochs, max sequence length 128.
2. Evaluate macro-F1 on your assigned partition's test split (same split as Task 1).
3. **Additional check:** re-run evaluation on your assigned partition's test tweets after applying Task 1's Dakshina-informed normalization at test time only (no retraining), and report whether normalizing the input helps or hurts a model fine-tuned on raw text.

Evaluation metrics to report:

- Macro-F1 on your assigned partition's test split (raw input).
- Macro-F1 on the same test split with Task-1-style normalization applied at test time only.

Task 3 — Combine & Improve: LID-Augmented MuRIL (40 marks)

Objective. Feed the Task 1 language-ID tagger's predictions into the Task 2 model as an auxiliary signal, on your assigned partition.

Step-by-step instructions:

1. Concatenate a learned language-tag embedding (dim = 16) to each subword's MuRIL embedding before the transformer layers, using the Task 1 LID tagger's per-token predictions (or, more simply, prepend a language-composition summary token – state which you implemented).
2. Apply the shared Dakshina-informed normalization to the input text before tokenization.
3. Fine-tune with the same hyperparameters as Task 2, on your assigned partition's train split.
4. Evaluate macro-F1 on your assigned partition's test split; compare against Task 1 and Task 2.

Evaluation metrics to report:

- Macro-F1 on your assigned partition's test split, compared against Task 1 and Task 2.

References

Patwa et al., *SemEval-2020 Task 9*, ACL Anthology 2020.semeval-1.100. Khanuja et al., arXiv:2103.10730 (MuRIL).

Deliverables and Submission Guidelines

Code

- Submit source code as either `.py` files or `.ipynb` Jupyter notebooks (or a mix). No other language or file format for code will be accepted.
- Organize code to mirror the Task 1 / Task 2 / Task 3 structure – three clearly separated modules or notebook sections, not one undifferentiated script, so each task's contribution is independently inspectable and runnable.
- All random seeds used for data splits, model initialization, and sampling must be fixed and stated in the code and the report.

Environment and Reproducibility

- **README.md**: environment setup; how to download/prepare all three datasets (script or exact instructions); how to run Task 1, Task 2, and Task 3 end-to-end, independently and in sequence; where results/checkpoints are written.
- **requirements.txt** (pip) *or* a conda `environment.yml` (choose one), pinning all package versions.
- Any pretrained checkpoints used (e.g. `bert-base-uncased`, `google/muril-base-cased`, GloVe vectors) must be named explicitly with source.

Report

A written report (PDF, 8–12 pages excluding references/appendix), covering, at minimum, for *every* task in the project you were assigned:

- **Method summary** for each of Task 1, Task 2, and Task 3, with any deviations from the specification explicitly justified (e.g. reduced model size due to compute constraints).
- **Quantitative analysis:** every metric specified under that task's *Evaluation metrics to report*, as tables/plots, with Task 1 and Task 2 always shown as reference points alongside Task 3's results – Task 3 is graded on the size and clarity of its improvement (or, if it does not improve, on the quality of the analysis of why not).
- **Qualitative analysis:** concrete examples (correct and incorrect) with discussion, for at least Task 2 and Task 3 – not just aggregate numbers.
- **Paper-reproduction comparison (Task 2):** a section explicitly comparing your reproduced numbers to the assigned paper's reported numbers, discussing any gap and its likely cause (data scale, compute budget, hyperparameter differences).
- **Discussion:** what worked, what didn't, and why – weighted at least as heavily as raw metric values.

Submission Package

A single archive (`.zip` or `.tar.gz`) containing: `code/` (subfolders `task1/`, `task2/`, `task3/`), `README.md`, `requirements.txt` or `environment.yml`, and `report.pdf`. Do not include large data files or model checkpoints – reference their download source in the README instead.